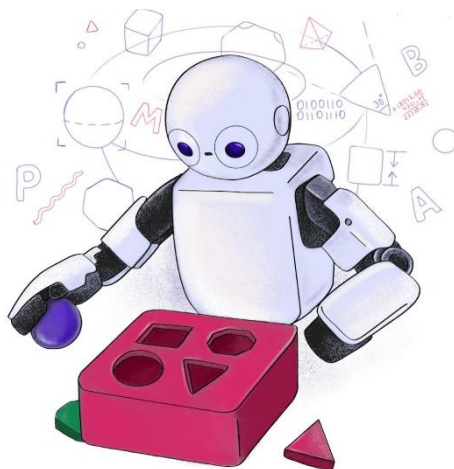


C24 - Inteligência Artificial: *Transformações dos dados*



DADOS BAGUNÇADOS: O caos que você conhece



DADOS PRONTOS PARA O MODELO: Música para os ouvidos da IA!



A jornada de todo cientista de dados: da bagunça à beleza

Continuamos na
jornada de
curadoria dos
dados.

O que veremos?

- Veremos algumas técnicas de transformação ou pré-processamento dos dados:
 - Divisão do *dataset*,
 - Escalonamento de atributos,
 - Codificação de atributos,
 - Engenharia de atributos,
 - Extração de atributos,
 - Seleção de atributos.

Antes de falarmos sobre as transformações, temos que falar sobre a divisão do *dataset* original

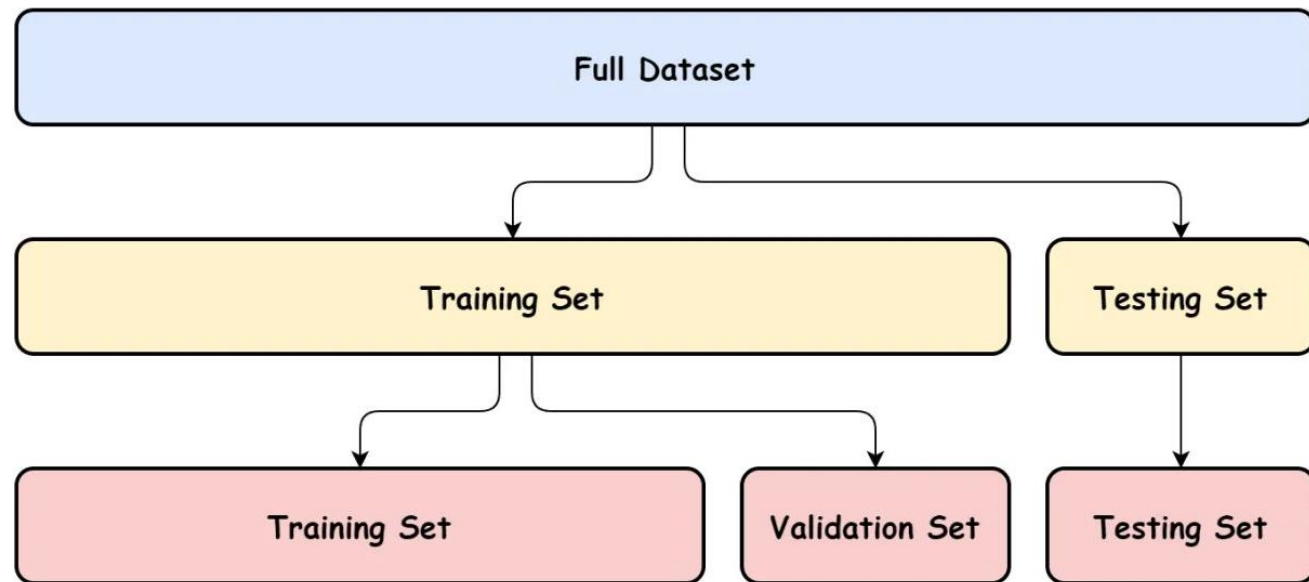


Não é correto usar o
mesmo conjunto de
dados para
treinamento, validação
e teste!

Por que não usar os mesmos dados para treino e teste

- Avaliar um modelo com os mesmos dados usados no treinamento **não mede sua capacidade real de generalização**.
- O resultado da avaliação pode ser **artificialmente otimista**, pois o modelo é testado com dados que já conhece.
 - Se o modelo memorizar os dados de treinamento (**overfitting**) em vez de aprender padrões gerais, o resultado da avaliação será excelente, mas o modelo pode não generalizar bem.
- Além disso, se o modelo for otimizado com o mesmo conjunto de treinamento, ocorrerá **vazamento de dados** (*data leakage*), comprometendo a avaliação do modelo.
 - O modelo será otimizado para o conjunto de treinamento, causando **overfitting**.

Divisão do *dataset*



- Como em ML, o objetivo é encontrar modelos que **generalizem** bem para dados novos (i.e., **inéditos**), precisamos definir uma forma de avaliar a generalização.
- Para isso, o conjunto total de amostras (*dataset*) é dividido em três subconjuntos **independentes**:
 - Treinamento
 - Validação
 - Teste

Divisão do *dataset*

- Todos os três devem ser **representativos do problema/fenômeno** sendo analisado.
 - **Representativos:** O *dataset* completo e, conseqüentemente, os subconjuntos devem **refletir** a **mesma distribuição** de dados do problema.
 - **Exemplo:** Se no *dataset* completo 70% das amostras são de *emails* legítimos e 30% são *spams*, os 3 subconjuntos devem manter estas porcentagens.
- Em geral, mas não é regra, divide-se o *dataset* original da seguinte forma:
 - 70%, 15%, 15%
 - 80%, 10%, 10%

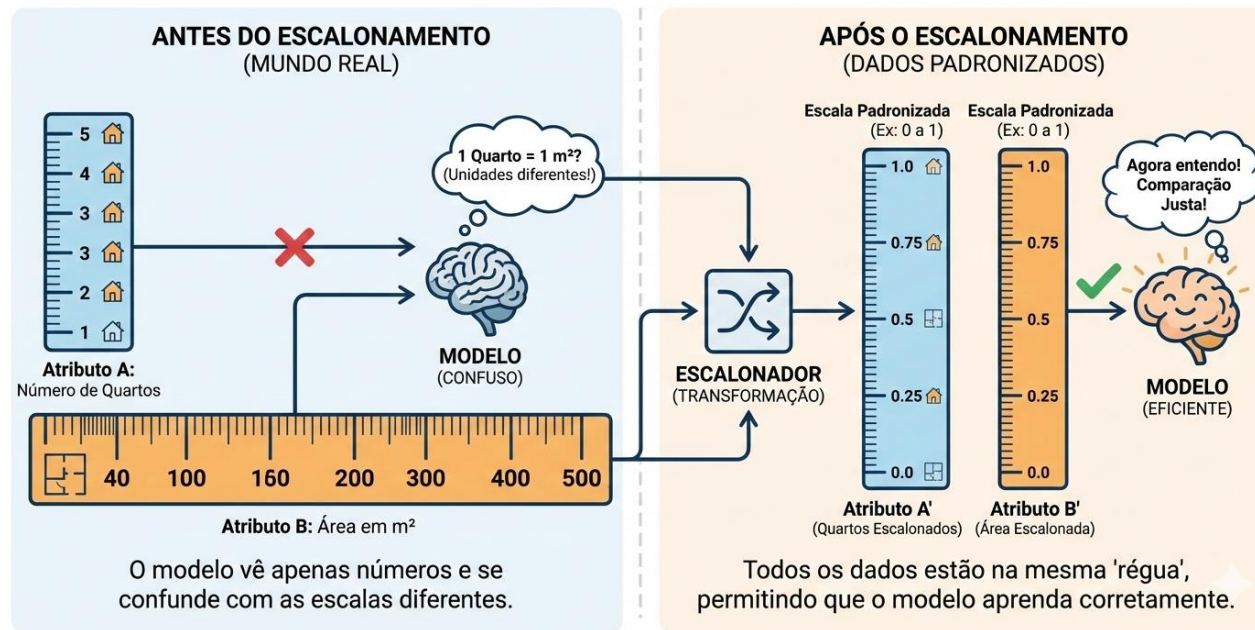
Divisão do *dataset*

- O conjunto de **treinamento** é para **treinar** os modelos de ML.
 - É onde o modelo “aprende” os padrões dos dados.
 - Quanto maior, mais diverso e variado, melhor o aprendizado.
- O conjunto de **validação** **avalia o modelo durante o desenvolvimento**.
 - Evita que o modelo decore os dados de treinamento (i.e., *overfit*).
 - Ajustar hiperparâmetros.
 - Escolher arquiteturas.
 - Decidir quando parar o treinamento (*early stopping*).
- O conjunto de **teste** **valida a capacidade de generalização** dos modelos.
 - Usado uma única vez, ao final.
 - Fornece uma estimativa imparcial do desempenho real do modelo.
 - Simula dados nunca vistos.



Agora veremos algumas transformações aplicadas aos dados que ajudam no aprendizado dos modelos, sendo a primeira delas, o **escalonamento**.

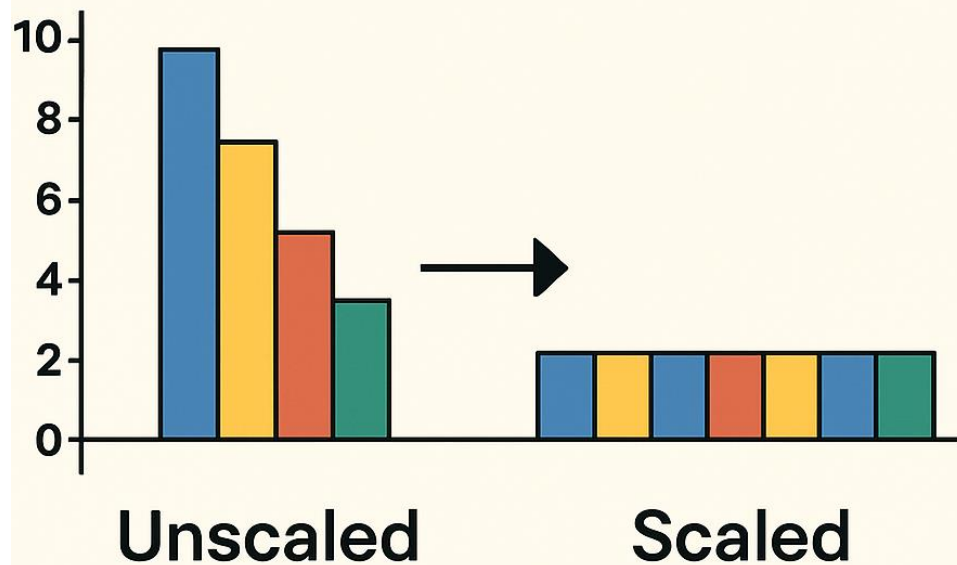
Escalonamento de atributos



- Precisamos colocar todos os dados na mesma régua (i.e., **escala**).
- Imaginem criar um modelo para a previsão do preço de casas.
 - **Atributo A:** Número de quartos (1 a 5).
 - **Atributo B:** Área em m^2 (40 a 500).
- Para o modelo, 1 quarto é igual a 1 m^2 .
 - Os modelos **não conhecem as unidades** do mundo real, eles **conhecem apenas números**!

Por que escalonar os atributos?

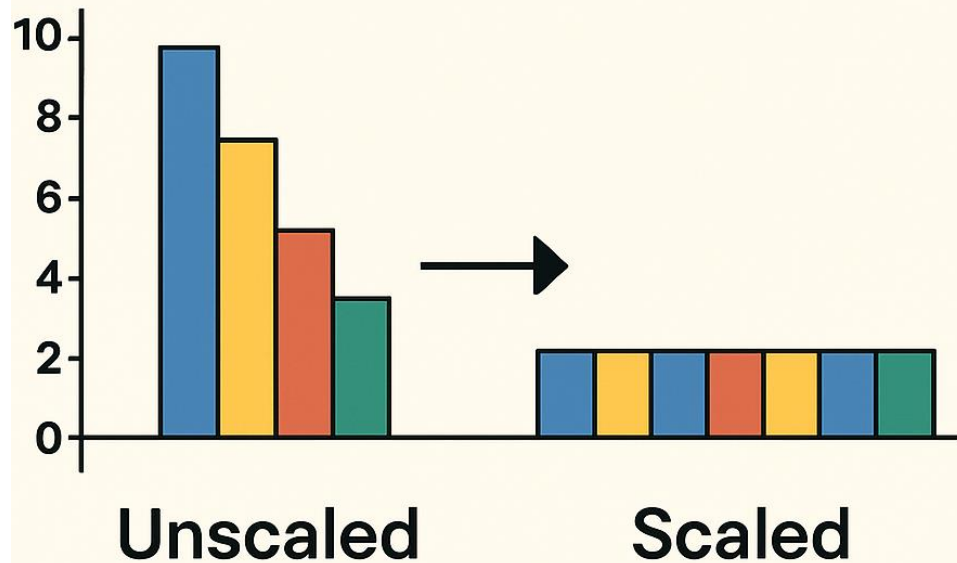
FEATURE SCALING



- Modelos de ML baseados em **distância** (e.g., KNN, SVM, PCA) ou **gradiente** (e.g., regressão linear/logística, redes neurais) **são sensíveis à escala dos atributos**.
- Atributos com **escala muito maior influenciam muito mais a previsão** dos modelos do que atributos com escala menor.
 - No exemplo da previsão do preço de casas, o modelo ignoraria os quartos e focaria só na área.

Por que escalonar os atributos?

FEATURE SCALING



- Mas e se todos os atributos são importantes para a previsão do modelo?
- O **escalonamento iguala a influência** dos atributos no modelo.
- Escalonamento transforma valores de atributos para uma mesma escala.
- Principais técnicas de escalonamento:
 - Padronização
 - Normalização
 - Escalonamento robusto

Normalização (*Min-Max Scaling*)

- **Como funciona:** Transforma todos os valores numéricos para uma **escala fixa, geralmente entre 0 e 1**.
- **Fórmula:**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \in [0, 1],$$

onde x é o valor original, x_{min} e x_{max} são os valores mínimo e máximo de um atributo (e.g., a coluna de um *dataset*).

Normalização (*Min-Max Scaling*)

- **Quando usar:**

- Dados sem *outliers* (analisar e remover antes),
- Com algoritmos que **não pressupõem** nenhuma **distribuição** (e.g., redes neurais, KNN),
- Com algoritmos **baseados em distância** (e.g., KNN, SVM, K-Means) e gradiente (Redes Neurais, Regressões).

- **Cuidado:** Muito sensível a *outliers*.

- Um *outlier* extremo “**esmaga**” todos os outros valores **para um intervalo muito pequeno próximo de zero**.
- Perde-se a **distinção entre valores típicos**.
- Isso reduz a eficácia do aprendizado, pois a **diferença entre os valores típicos é praticamente eliminada** na escala normalizada.

Normalização (*Min-Max Scaling*)

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Instância do normalizador com intervalo de 0 a 1  
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
# Aprende os parâmetros de normalização (min e max) no  
treino
```

```
X_scaled = scaler.fit_transform(X_train)
```

```
# Aplica o scaler aprendido ao conjunto de teste
```

```
X_test_scaled = scaler.transform(X_test)
```

Por que não calculamos os
parâmetros de
escalonamento usando o
conjunto completo?

Para não haver vazamento de informações do “futuro” no treinamento.

Padronização (*Standard Scaling*)

- **Como funciona:** Transforma os atributos para terem **média 0 e desvio padrão 1**, **não impondo limites**.

- **Fórmula:**

$$x' = \frac{x - \mu_x}{\sigma_x},$$

onde x é o valor original, μ_x e σ_x são a média e o desvio padrão de um atributo.

- **Vantagens:**

- Torna os dados "comparáveis" em termos do número de desvios padrão da média.
- **Mais robusto** a *outliers*, mas não a *outliers* extremos.
 - Por não dividir os valores por um denominador muito grande, no caso da presença de *outliers*, não os transforma em valores próximos de zero e, portanto, não se perde a distinção entre eles.

Padronização (Standard Scaling)

- **Quando usar:**
 - Em geral, com algoritmos que assumem distribuição "normal" ou Gaussiana (e.g., GNB, GMM, alguns modelos para detecção de anomalias), e
 - quando os dados possuem *outliers*, mas não extremos.
- **Cuidado:** Se os dados originais são assimétricos (i.e., cauda longa), eles continuarão assimétricos, apenas centralizados e com variância igual a 1.
- **Importante:** Se o conjunto de dados tiver *outliers extremos*, o **escalador robusto é uma opção superior** à padronização, pois ignora os valores discrepantes no cálculo do escalonamento.
 - *Outliers extremos* podem **influenciar a média e a variância** dos atributos de forma negativa.

Padronização (Standard Scaling)

```
from sklearn.preprocessing import StandardScaler
```

```
# Instância do padronizador
```

```
scaler = StandardScaler()
```

```
# Aprende os parâmetros de padronização no treino
```

```
X_scaled = scaler.fit_transform(X_train)
```

```
# Aplica o scaler aprendido no teste
```

```
X_test_scaled = scaler.transform(X_test)
```

Escalonamento robusto (Robust Scaling)

- **Como funciona:** Usa **mediana e distância interquartil** no escalonamento.
- **Fórmula:**

$$x' = \frac{x - \text{mediana}}{\underbrace{Q_3 - Q_1}_{\text{IQR}}},$$

onde x é o valor original e IQR é a distância interquartil de um atributo.

- **Vantagem:** Por utilizar a mediana e o IQR em vez da média e do desvio padrão, os **outliers têm influência mínima** no escalonamento.
 - Mantém a relação de distância entre os valores típicos **melhor que a padronização quando existem outliers extremos**.
 - Ou seja, não deixa que *outliers* extremos “esmaguem” os valores típicos em valores próximos de 0.

Escalonamento robusto (Robust Scaling)

- **Quando usar:**

- Quando há *outliers* significativos, mas desejamos mantê-los sem impactar o escalonamento.
- Quando as distribuições são assimétricas (i.e., com cauda longa).

- **Cuidado:**

- Como, em geral, **não se remove** os *outliers*, algoritmos **sensíveis à distância** (e.g., KNN, Regressores) podem ter seu **aprendizado impactado**.
- **Preserva a escala** dos dados típicos, **mas não elimina nem disfarça** os *outliers*.

Escalonamento robusto (Robust Scaling)

```
from sklearn.preprocessing import RobustScaler

# Instância do escalonador robusto
scaler = RobustScaler()

# Aprende os parâmetros de escalonamento no treino
X_scaled = scaler.fit_transform(X_train)

# Aplica o scaler aprendido no teste
X_test_scaled = scaler.transform(X_test)
```

Armadilhas comuns durante o escalonamento

- Para evitar **vazamento de dados**, o escalonador deve sempre ser **ajustado no conjunto de treinamento**.
- Os conjuntos de validação e teste, são **transformados com os parâmetros encontrados com o conjunto de treinamento**.
- Geralmente, **não se aplica escalonamento à variável alvo**, i.e., os rótulos, apenas aos atributos.
- Em geral, **não se escala atributos binários ou one-hot** sem necessidade, pois pode-se criar valores fracionários desnecessários.

Codificação de atributos categóricos

- **Motivação:** Modelos de ML não entendem texto, portanto, precisamos converter categorias em números.
- As principais técnicas de codificação são:
 - One-Hot encoding
 - Label encoding
 - Target encoding
 - Binary encoding
- **OBS.:** A codificação pode ser considerada uma forma de **engenharia de atributos**.
 - Processo de **criar atributos** relevantes a **partir dos existentes** para melhorar o desempenho de um modelo de ML.

One-Hot encoding

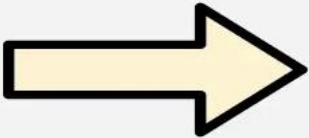


Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- Usada com atributos nominais (i.e., sem ordem) e com baixa cardinalidade (i.e., com poucas categorias distintas).
- **Como funciona:** Cria uma nova **coluna binária** para **cada categoria** distinta.
- **Limitação:** Gera alta dimensionalidade (i.e., muitas colunas) se houver muitas categorias.
 - Pode criar o problema conhecido como a “**maldição da dimensionalidade**”.
 - Pontos ficam muito distantes (perde-se noção de similaridade), dificulta a visualização e aumenta o custo computacional.

Label e ordinal encoding



The diagram illustrates the process of label encoding. On the left, a table with a 'Color' header contains the values 'Red', 'Green', 'Blue', and 'Red' in its rows. A large yellow arrow points to the right, where a second table is shown. This second table also has a 'Color' header, but its rows contain the numerical values '0', '1', '2', and '0' respectively, representing the categorical values from the first table.

Color
Red
Green
Blue
Red

Color
0
1
2
0

- Converte **cada categoria em um número inteiro único**, considerando (*ordinal*) ou não uma ordem (*label*).
- *Label* é usada com rótulos em problemas de classificação e *ordinal* com variáveis ordinais.
 - Para tamanho de camisetas (P, M, G): *ordinal*.
 - Para animais e cores: *label/ordinal*.
- **Risco:** *Label* pode introduzir uma **relação de ordem falsa** que afeta negativamente algoritmos ***sensíveis a distâncias*** (e.g., regressores e KNN).

Target encoding

workclass	target		workclass	target mean		workclass
State-gov	0		State-gov	0		0
Self-emp-not-inc	1	→	Self-emp-not-inc	1		1
Private	0		Private	1/3	→	1/3
Private	0					1/3
Private	1					1/3

Em vez de usar números arbitrários (**Label Encoding**) ou criar várias colunas (**One-Hot Encoding**), **usa informação estatística do próprio problema**.

- Substitui cada categoria pela **média do valor do alvo** correspondente a essa categoria.
- **Quando usar:** Ideal para atributos de **alta cardinalidade**, pois não aumenta a dimensionalidade do *dataset* e quando há **relação forte entre a categoria e o *target***.
- **Risco:** *Overfitting* por *data leakage* e por categorias com poucas amostras.
 - Se for calculado **usando todo o dataset**.
 - Categorias com poucos exemplos produzem **estimativas de média muito ruidosas**.

Binary encoding

Color		Color_0	Color_1
Green	Binary encoding →	0	0
Red		0	1
Black		1	0
Orange		1	1

- Combina os benefícios das codificações *One-Hot* e *Label*.
- Cada **categoria recebe um número inteiro (Label)** que é **convertido para sua representação binária**, criando colunas para cada bit.
- **Quando usar:** Ótimo para variáveis categóricas de alta cardinalidade.
- **Desvantagem:** Por algumas categorias **compartilharem bits, modelos lineares podem achar que existe uma relação entre elas**. Modelos de árvore lidam bem com isso.

Exemplos de *encoding*

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder,  
OrdinalEncoder, TargetEncoder  
  
# SciKit-Learn não implementa o codificador binário (pip install  
category_encoders)  
  
from category_encoders import BinaryEncoder  
  
# LabelEncoder(), LabelEncoder(), TargetEncoder() ou BinaryEncoder()  
enc = OneHotEncoder()  
enc.fit_transform(X_train)  
  
enc.transform(X_test)  
enc.inverse_transform(X_test)
```

Engenharia de atributos

- **Definição:** É a transformação dos atributos originais em atributos, mais úteis e informativos.
- **Por que fazer?** Ajuda o modelo a enxergar relações que não estão óbvias.
- **Objetivo:** Melhorar a capacidade de predição (generalização) do modelo.
- **Exemplos:**
 - Data de nascimento → Idade, faixa etária.
 - Coordenadas GPS → Distância até certo ponto.
 - Preço e quantidade → Valor total da venda.
 - *Timestamp* → Hora, dia da semana, feriado?
 - Comprimento e largura → Área.
 - Atributos → Interação entre atributos: Polinômios (e.g., $x'_1 = x_1x_2 + x_1^2x_2$).

Engenharia de atributos

```
import pandas as pd
```

```
# Exemplo de dados
```

```
df = pd.DataFrame({  
    "preco": [10, 20, 15],  
    "quantidade": [2, 1, 3],  
    "desconto": [0, 5, 0]})
```

```
# Feature Engineering
```

```
df["valor_total"] = df["preco"] * df["quantidade"]  
df["houve_desconto"] = (df["desconto"] > 0).astype(int)
```

Extração de atributos

- **Definição:** É o processo de **transformar dados** em um **conjunto menor de atributos significativos** que podem ser **processados de forma mais eficiente, mantendo as informações essenciais** (em geral, perde-se parte da informação).
 - Transforma os dados em uma representação mais eficiente, em geral, de menor dimensão.
- **Por que fazer?**
 - Reduz a dimensionalidade,
 - Remove redundância,
 - Reduz os recursos computacionais necessários para armazenamento e treinamento,
 - Acelera o treinamento,
 - Melhora o desempenho do modelo, pois os dados são mais informativos e menos ruidosos.

Técnicas comuns para extração de atributos

- **Dados tabulares:** Análise de componentes principais (PCA) e análise de componentes independentes (ICA).
 - Criam novos **atributos não correlacionados** que **capturam as direções de variância máxima** (PCA) ou **atributos que são independentes** (ICA).
- **Texto:** *Bag of words* (BoW), *Term Frequency-Inverse Document Frequency* (TF-IDF), *Word Embeddings*.
 - **Transformam texto em representações numéricas** (modelos de ML entendem números), variando da **frequência de palavras** até **vetores que capturam significado e contexto**.

Técnicas comuns para extração de atributos

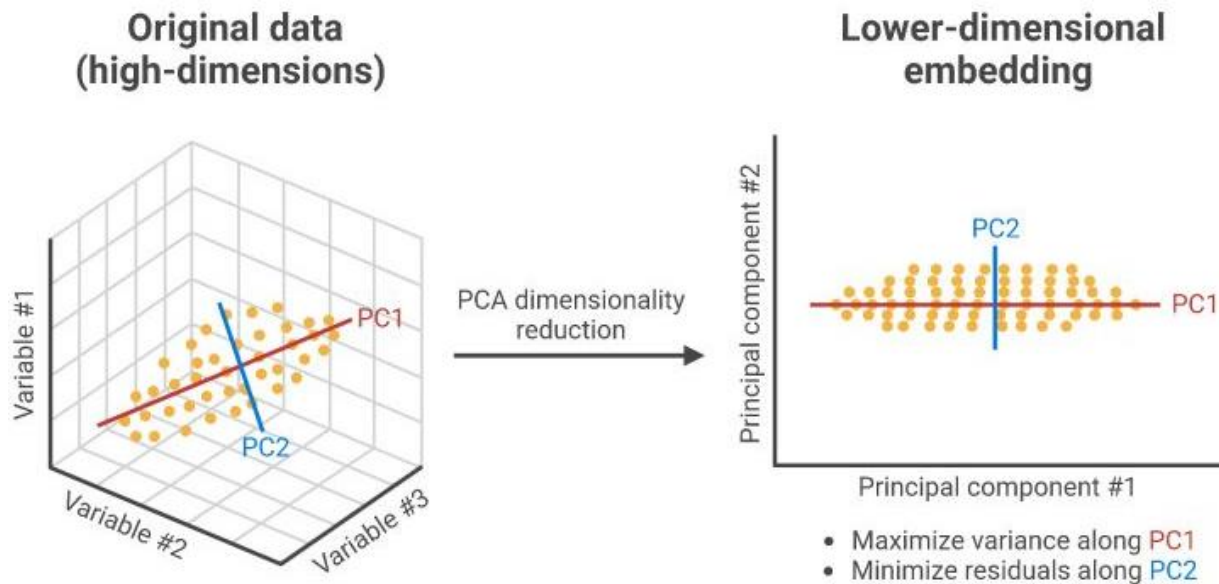
- **Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*
 - Transformam **pixels em representações numéricas que capturam padrões visuais**, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.
- **Séries temporais (áudio):** *Mel-frequency cepstral coefficients (MFCCs), Transformada rápida de Fourier (FFT), Espectogramas, Transformada de Wavelet.*
 - Transformam os sinais em **características que capturam propriedades espectrais ou de tempo-frequência**.



iStock™
Credit: meltonmedia

O PCA projeta dados de alta dimensão em um espaço de menor dimensão.

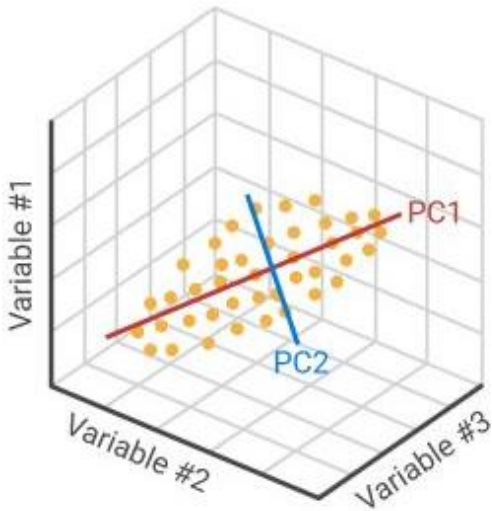
Análise de componentes principais (PCA)



- PCA encontra as **direções (componentes principais)** onde os **dados mais variam** e **projeta** os dados nessas direções.
- Ele encontra as **direções que descrevem como os dados estão espalhados** (i.e., máxima variância).
- O objetivo é **reduzir a dimensionalidade** com **mínima perda de informação**.
- Aplicado através de **transformação matricial**.

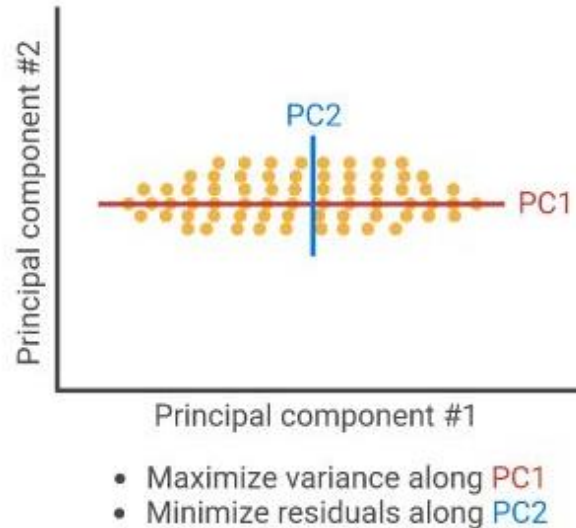
Análise de componentes principais (PCA)

Original data
(high-dimensions)



PCA dimensionality
reduction

Lower-dimensional
embedding



- Na figura, **PC1** é a direção onde os **dados variam mais**.
- Já **PC2** é a direção **ortogonal** à PC1 com a **segunda maior variância**.
- Mantendo **PC1**, **preserva-se a maior parte da informação**.
- PC2 representa variação residual (menor importância).

Análise de componentes principais (PCA)

- As PCs são ordenadas por variância.
 - A matriz de **projeção** é ordenada da **componente de maior variância para a menor**.
- O PCA **assume relações lineares entre os atributos** (traça linhas retas para capturar as variâncias).
 - Se os dados **não são lineares** (como uma senoide, meia lua ou um círculo), o **PCA vai ter um desempenho bem ruim**.
 - Nestes casos, deve-se usar técnicas não lineares como Kernel PCA e t-SNE.
- Deve ser **aplicado após a separação do dados** em conjuntos de treinamento, validação e teste para evitar vazamento de dados.
- Os **dados devem ser padronizados** antes de sua aplicação.
 - Sem padronização, **atributos com maior escala dominam** as componentes principais.

Análise de componentes principais (PCA)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Padronizar os dados antes do PCA
```

```
X_train_scaled = StandardScaler().fit_transform(X_train)
```

```
X_test_scaled = StandardScaler().transform(X_test)
```

```
# Extrair 2 componentes
```

```
pca = PCA(n_components=2)
```

```
X_train_pca = pca.fit_transform(X_train_scaled)
```

```
X_test_pca = pca.transform(X_test_scaled)
```

```
print(pca.explained_variance_ratio_)
```

OBS.: “n_components” pode ser também um número entre 0 e 1 especificando o mínimo de variância explicada desejada.

Se não for definido, todas as components são usadas.

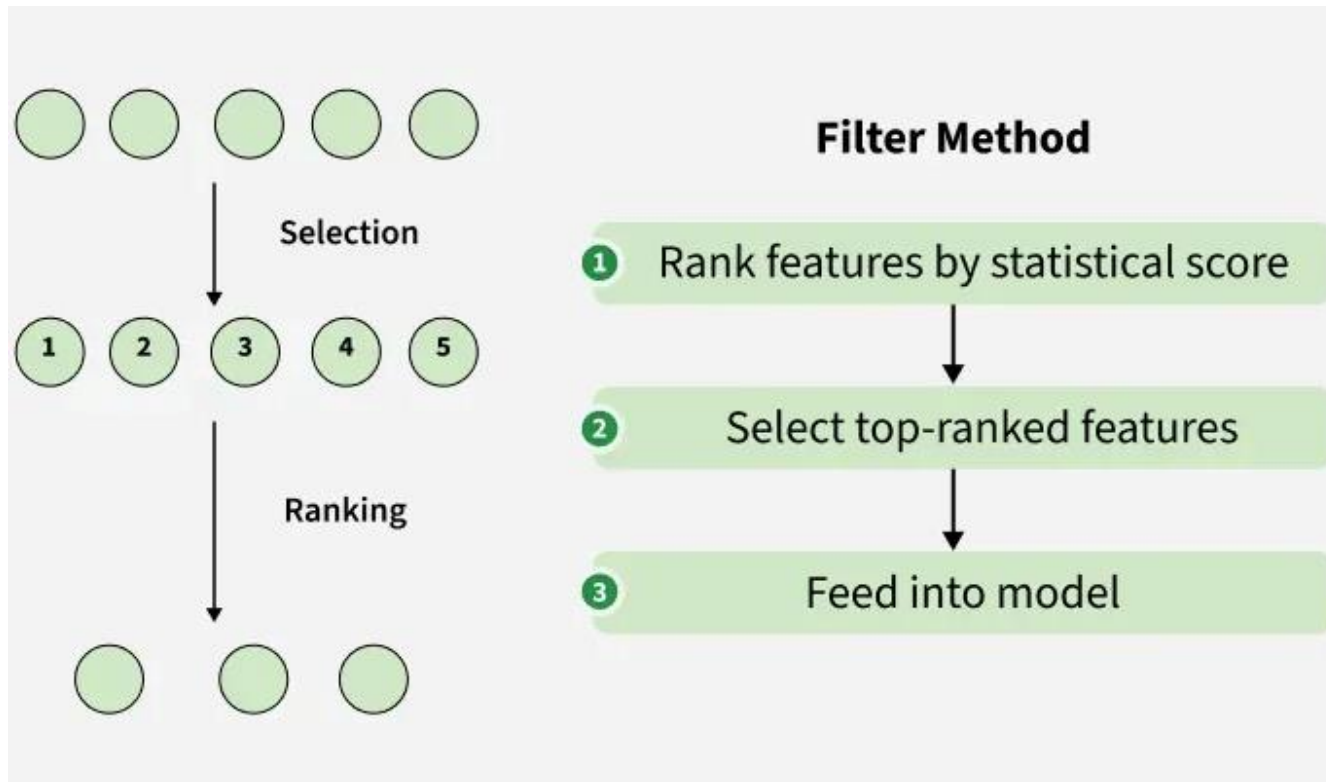
Seleção de atributos

- **Definição:** É o processo de **selecionar um subconjunto de atributos** que contribuem para a **melhoria do desempenho do modelo**.
- Por que usar?
 - **Reduz *overfitting*:** **Modelos mais simples** com menos atributos tendem a **generalizar melhor** para novos dados.
 - **Reduz custo computacional:** **Menos dados** significam **algoritmos** que processam **mais rápido**.
 - **Facilita interpretação:** É mais **fácil explicar** um modelo que utiliza 5 variáveis do que um que utiliza 500.

Seleção de atributos

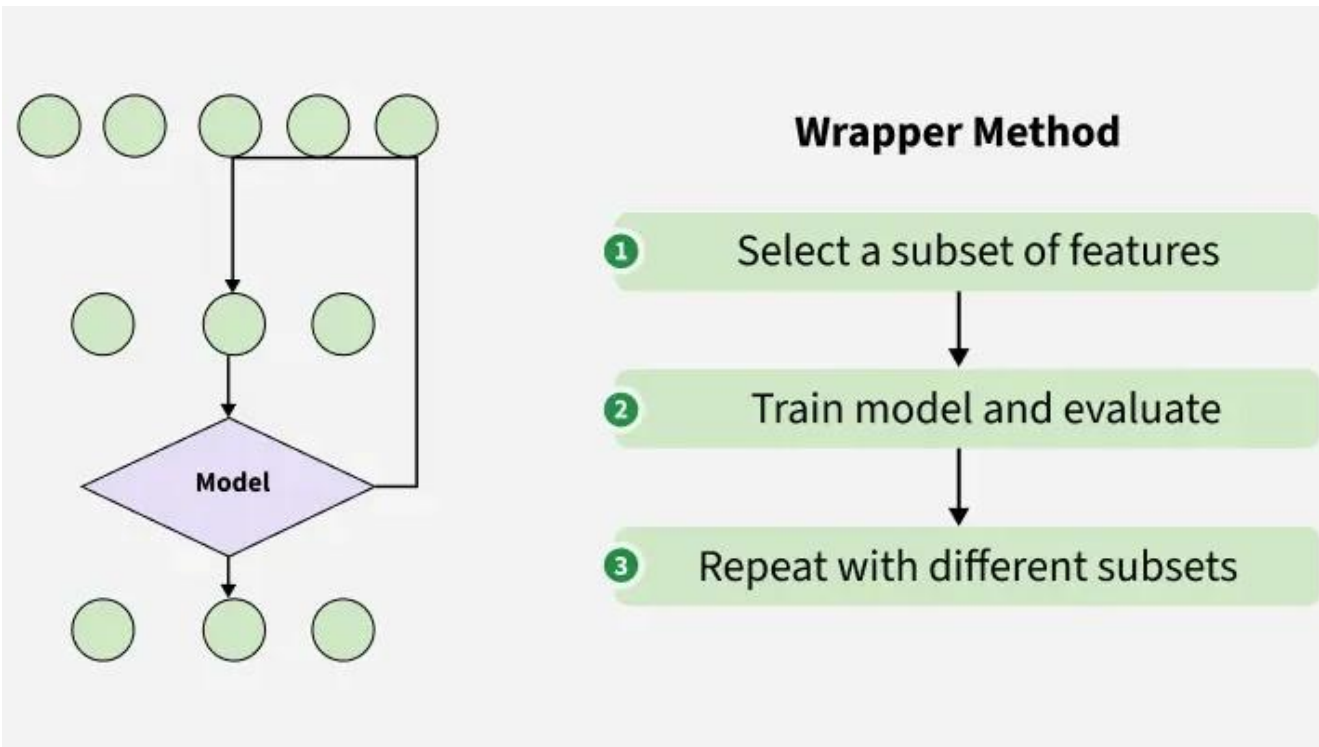
- Existem três tipos principais de técnicas de seleção de atributos:
 - **Filtragem:** Baseados em **medidas estatísticas dos dados**, como a correlação.
 - **Exemplos:** Correlação de Pearson/Spearman, Teste qui-quadrado, limiar de variância, ganho de informação, informação mútua.
 - **Encapsulamento:** **Usa o próprio modelo de ML para testar subconjuntos** de variáveis e avaliar a performance.
 - **Exemplo:** **Eliminação recursiva** de atributos.
 - **Embarcado:** A **seleção ocorre durante o processo de treinamento** do próprio algoritmo. É **integrada** (i.e., embarcada) no **treinamento**.
 - **Exemplos:** Regressão LASSO e árvores de decisão.
- Filtragem usa apenas informações do *dataset*, já o encapsulamento e o embarcado, usam informações do *dataset* e do modelo para selecionar os atributos.

Métodos de filtro



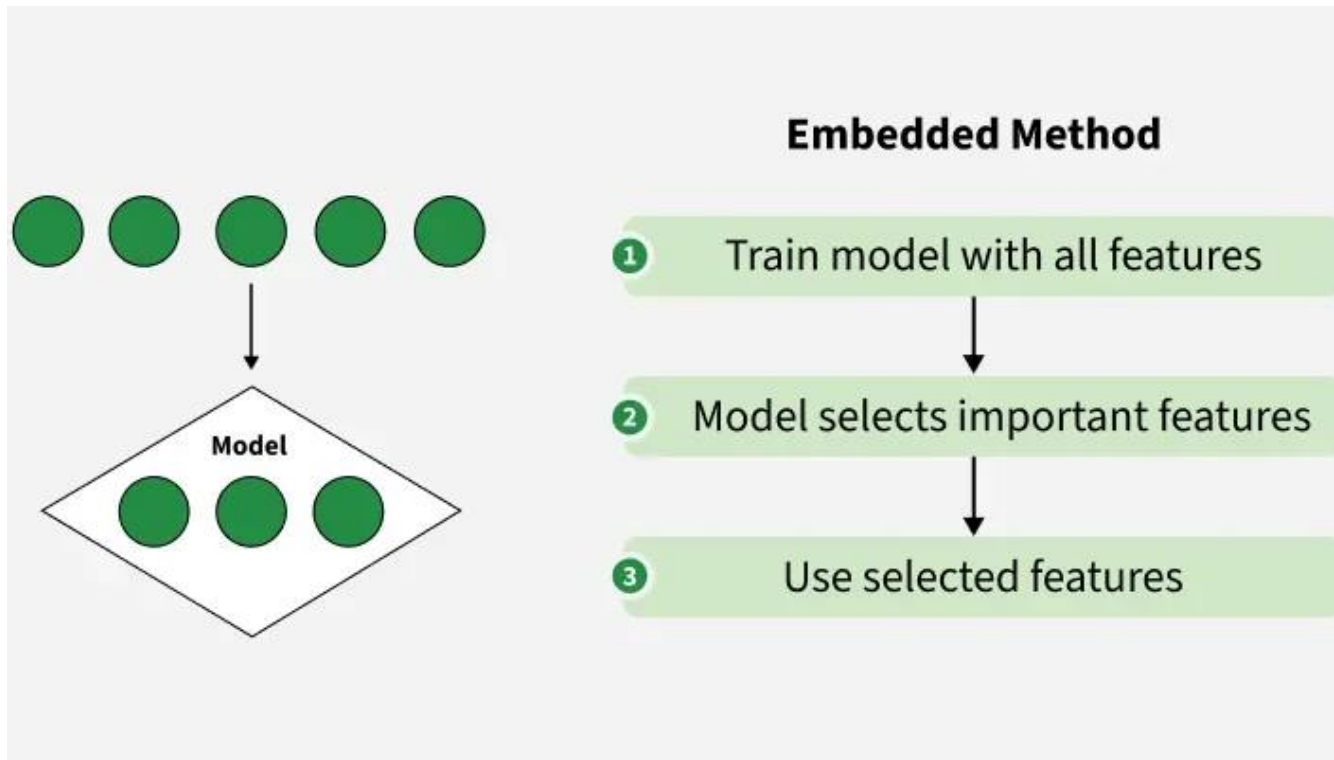
- São rápidos e independentes do modelo de ML.
- Escala bem para *datasets* grandes.
- Filtros simples podem **ignorar interações relevantes** entre atributos (e.g., usar apenas a **correlação entre atributo e alvo**).
- Dependem da escolha adequada da métrica (e.g., relações lineares e não lineares).

Métodos de encapsulamento



- **Otimizam** os atributos **para um modelo específico**.
- Em geral, tem **desempenho superior** aos métodos de **filtragem**.
- Porém, são **computacionalmente caros** e **suscetíveis ao *overfitting***, pois podem **ajustar excessivamente** os **atributos** a um **modelo específico**.

Métodos embarcados



- Seleccionam atributos **durante o treinamento** do modelo.
- Apresenta **equilíbrio** entre **desempenho e tempo para seleção**.
- Porém, têm **interpretabilidade limitada** (por que um atributo foi removido?) e **não são suportados por todos os algoritmos**.

Seleção de atributos na biblioteca SciKit-learn

- O módulo `sklearn.feature_selection` implementa vários métodos de seleção de atributos, dentre eles podemos citar:
 - Filtragem
 - `VarianceThreshold`: Elimina atributos com variância abaixo do limiar configurado.
 - `SelectKBest`: Seleciona os K melhores atributos de acordo com um *score* definido (e.g., informação mútua, ganho de informação, etc.)
 - Encapsulamento
 - `RFE`: Elimina os atributos de forma recursiva até que apenas as melhores permaneçam.
 - Embarcado
 - `SelectFromModel`: Seleção usando modelos com penalidade L1 ou baseados em árvore, como LASSO e árvores de decisão.

Seleção de atributos na biblioteca SciKit-learn

```
from sklearn.feature_selection import VarianceThreshold

# Remove features com variância <= 0.1
selector = VarianceThreshold(threshold=0.1)

X_train_reduced = selector.fit_transform(X_train)
X_test_reduced = selector.transform(X_test)
```

Extração vs. seleção de atributos

- A extração **transforma** os **atributos originais para criar novos**, geralmente reduzindo a dimensionalidade.
- A seleção **escolhe** um **subconjunto dos atributos existentes** mais relevantes **sem alterá-los**.

Exemplo

- [Exemplo: pre_processing.ipynb](#)



Lista de exercícios

Lista #4 – Transformações

Perguntas?

Obrigado!